HARMONY_OS_GUIDE

HelixCode for Harmony OS - Complete User Guide

Table of Contents

Introduction

HelixCode for Harmony OS is a specialized client optimized for Huawei's Harmony OS platform, featuring distributed computing capabilities, cross-device synchronization, and native integration with Harmony OS ecosystem features.

Key Features

- **Distributed Computing Engine**: Execute AI workloads across multiple Harmony OS devices
- **Cross-Device Sync**: Seamless data synchronization across your Super Device ecosystem
- **AI Acceleration**: Native support for NPU and GPU acceleration
- **Multi-Screen Collaboration**: Work across multiple screens and devices simultaneously

- **Resource Optimization**: Intelligent resource management for optimal performance
- **Service Coordination**: Automatic service discovery and failover support

---

System Requirements

Minimum Requirements

- **Operating System**: Harmony OS 3.0 or later
- **RAM**: 2 GB minimum (4 GB recommended)
- **Storage**: 200 MB free space
- **Network**: Active network connection for distributed features
- **Database**: PostgreSQL 13+ (for standalone mode)

Recommended Requirements

- **Operating System**: Harmony OS 4.0+
- **RAM**: 8 GB or more
- **Storage**: 1 GB free space (for caching and logs)
- **Network**: High-speed connection (1 Gbps+) for distributed computing
- **Database**: PostgreSQL 15+ with connection pooling
- **Redis**: Redis 6.0+ for caching (optional but recommended)
- **NPU**: Kirin NPU or compatible for AI acceleration

Hardware Acceleration Support

- **NPU**: Kirin 990, 9000 series or later
- **GPU**: Mali-G78 or later for graphics acceleration
- **DSP**: Hardware DSP support for audio/video processing

Installation

Quick Installation (Automated)

The easiest way to install HelixCode on Harmony OS is using the automated deployment script:

```
# Build the binary
cd HelixCode
make harmony-os
```

```
# Deploy with automated script (requires root)
sudo ./scripts/deploy-harmony-os.sh
```

The script will: - Install the binary to `/opt/helixcode/` - Create default configuration at `/etc/helixcode/harmony-config.yaml` - Set up systemd service (if available) or init script - Create helixcode user and set appropriate permissions - Configure logging and data directories

Manual Installation

If you prefer manual installation:

Step 1: Build the Binary

```
cd HelixCode
make harmony-os
```

This creates `bin/harmony-os` (approximately 56 MB).

Step 2: Install Binary

```
sudo mkdir -p /opt/helixcode
sudo cp bin/harmony-os /opt/helixcode/
sudo chmod +x /opt/helixcode/harmony-os
sudo ln -s /opt/helixcode/harmony-os /usr/local/bin/helixcode-
    harmony
```

Step 3: Create Configuration

```
sudo mkdir -p /etc/helixcode
sudo mkdir -p /var/log/helixcode
sudo mkdir -p /var/lib/helixcode

# Create config file (see Configuration section below)
sudo nano /etc/helixcode/harmony-config.yaml
```

Step 4: Create System User

```
sudo useradd -r -s /bin/false -d /var/lib/helixcode helixcode
sudo chown -R helixcode:helixcode /var/log/helixcode
sudo chown -R helixcode:helixcode /var/lib/helixcode
```

Step 5: Configure Service

For systemd-based systems:

```
sudo cp scripts/helixcode-harmony.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable helixcode-harmony
sudo systemctl start helixcode-harmony
```

Configuration

Basic Configuration

Create `/etc/helixcode/harmony-config.yaml`:

```yaml
# HelixCode Harmony OS Configuration

server:
  address: "0.0.0.0"
  port: 8080

database:
  host: "localhost"
  port: 5432
  user: "helixcode"
  dbname: "helixcode"
  # Set password via HELIX_DATABASE_PASSWORD environment variable

redis:
  enabled: true
  host: "localhost"
  port: 6379
  # Set password via HELIX_REDIS_PASSWORD environment variable

auth:
  token_expiry: 86400      # 24 hours
  session_expiry: 604800   # 7 days

harmony:
  # Distributed computing
  enable_distributed_computing: true
  enable_cross_device_sync: true
  sync_interval: 30  # seconds

  # Resource management
  enable_resource_optimization: true
  enable_ai_acceleration: true
```

```yaml
  gpu_enabled: true
  npu_enabled: true

  # System integration
  enable_system_monitoring: true
  enable_multi_screen: true
  enable_super_device: true

  # Service coordination
  service_discovery_enabled: true
  service_failover_enabled: true
  health_check_interval: 15  # seconds

workers:
  health_check_interval: 30
  max_concurrent_tasks: 20
  distributed_mode: true

tasks:
  enable_distributed_execution: true
  checkpoint_interval: 300
  max_retries: 3

logging:
  level: "info"
  file: "/var/log/helixcode/harmony-os.log"
  enable_distributed_logging: true
```

Environment Variables

Create `/etc/helixcode/harmony.env`:

```
# Database
HELIX_DATABASE_PASSWORD=your_secure_password

# Redis (if enabled)
HELIX_REDIS_PASSWORD=your_redis_password

# JWT Secret
HELIX_AUTH_JWT_SECRET=your_jwt_secret_key

# Harmony OS specific
HARMONY_DEVICE_ID=auto
HARMONY_SUPER_DEVICE_ENABLED=true
HARMONY_NPU_ENABLED=true
```

Advanced Configuration

Distributed Computing Settings

```yaml
harmony:
  distributed:
    # Task distribution strategy
    distribution_strategy: "load_balanced"  # or "round_robin",
        "capability_based"

    # Task scheduling
    scheduler_interval: 5  # seconds
    max_queue_size: 1000

    # Network settings
    peer_discovery_port: 8081
    data_transfer_port: 8082
    compression_enabled: true

    # Performance tuning
    batch_size: 10
    prefetch_tasks: true
    prefetch_count: 5
```

AI Acceleration Settings

```yaml
harmony:
  ai_acceleration:
    # NPU settings
    npu_enabled: true
    npu_model_cache: "/var/lib/helixcode/npu_cache"
    npu_max_memory: "2GB"

    # GPU settings
    gpu_enabled: true
    gpu_memory_fraction: 0.8
    gpu_allow_growth: true

    # Model optimization
    enable_quantization: true
    enable_pruning: false
    precision: "fp16"  # or "fp32", "int8"
```

Core Features

Starting the Application

```
# Start as system service
sudo systemctl start helixcode-harmony

# Or run directly
helixcode-harmony

# Run with custom config
helixcode-harmony --config /path/to/config.yaml
```

Main Interface

The Harmony OS client provides a native Fyne-based GUI with four main tabs:

1. **Dashboard**: System overview, task status, and resource usage
2. **Tasks**: Task management and execution monitoring
3. **Workers**: Distributed worker pool management
4. **Settings**: Configuration and preferences

Basic Operations

Creating a Task

```
// Via CLI
helixcode-harmony task create \
  --name "Code Analysis" \
  --type "analysis" \
  --priority "high" \
  --distributed

// Via API
curl -X POST http://localhost:8080/api/tasks \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "name": "Code Analysis",
    "type": "analysis",
    "priority": "high",
    "distributed": true
  }'
```

Monitoring Task Progress

```
# List all tasks
helixcode-harmony task list

# Get task details
helixcode-harmony task get <task-id>

# Stream task logs
helixcode-harmony task logs <task-id> --follow
```

Distributed Computing

Overview

The Harmony OS client includes a powerful distributed computing engine that can execute tasks across multiple Harmony OS devices in your ecosystem.

Setting Up Distributed Computing

1. Enable on Primary Device

Edit `/etc/helixcode/harmony-config.yaml`:

```yaml
harmony:
  enable_distributed_computing: true
  distributed:
    role: "primary"  # This is the coordinator
    discovery_enabled: true
    auto_accept_workers: false  # Require manual approval
```

2. Configure Worker Devices

On secondary devices:

```yaml
harmony:
  enable_distributed_computing: true
  distributed:
    role: "worker"
    primary_host: "192.168.1.100"  # IP of primary device
    auto_register: true
```

3. Start Distributed Engine

```
# On primary device
helixcode-harmony distributed start --role primary

# On worker devices
helixcode-harmony distributed start --role worker --primary
        192.168.1.100
```

Task Distribution Strategies

Load Balanced

Tasks are distributed based on current load:

```yaml
harmony:
  distributed:
    distribution_strategy: "load_balanced"
    load_metrics:
      - cpu
      - memory
      - gpu_utilization
```

Capability Based

Tasks are assigned based on device capabilities:

```yaml
harmony:
  distributed:
    distribution_strategy: "capability_based"
    required_capabilities:
      npu: true
      min_memory: 4096  # MB
      gpu_memory: 2048  # MB
```

Round Robin

Simple round-robin distribution:

```yaml
harmony:
  distributed:
    distribution_strategy: "round_robin"
```

Monitoring Distributed Tasks

```
# View cluster status
helixcode-harmony distributed status

# List all workers
helixcode-harmony distributed workers

# View task distribution
helixcode-harmony distributed tasks

# Check worker health
helixcode-harmony distributed health <worker-id>
```

Cross-Device Synchronization

Overview

HelixCode for Harmony OS supports automatic synchronization across devices in your Super Device ecosystem.

Enabling Sync

```
harmony:
  enable_cross_device_sync: true
  sync_interval: 30  # seconds
  sync_mode: "incremental"  # or "full"

  sync_items:
    - tasks
    - sessions
    - configurations
    - logs
```

Sync Modes

Real-Time Sync

Data is synchronized immediately on changes:

```
harmony:
  sync_mode: "realtime"
  sync_debounce: 1000  # ms
```

Periodic Sync

Data is synchronized at regular intervals:

```yaml
harmony:
  sync_mode: "periodic"
  sync_interval: 30  # seconds
```

Manual Sync

Synchronization only happens when triggered:

```
helixcode-harmony sync now
```

Conflict Resolution

```yaml
harmony:
  sync_conflict_resolution: "last_write_wins"
  # Options: last_write_wins, merge, manual

  # For merge strategy
  merge_strategy:
    tasks: "append"
    configs: "deep_merge"
    sessions: "latest"
```

Monitoring Sync Status

```
# Check sync status
helixcode-harmony sync status

# View sync history
helixcode-harmony sync history

# Force full sync
helixcode-harmony sync full --force
```

AI Acceleration

NPU Acceleration

The Kirin NPU provides hardware acceleration for AI workloads.

Enabling NPU

```yaml
harmony:
  enable_ai_acceleration: true
  npu_enabled: true
  npu_config:
    device: "npu0"
    max_batch_size: 8
    precision: "fp16"
    cache_models: true
```

Using NPU for Inference

```
# Run inference with NPU
helixcode-harmony infer \
  --model llama-3-8b \
  --device npu \
  --batch-size 4

# Check NPU utilization
helixcode-harmony device npu-status
```

GPU Acceleration

Enabling GPU

```yaml
harmony:
  gpu_enabled: true
  gpu_config:
    device_id: 0
    memory_fraction: 0.8
    allow_growth: true
```

GPU Memory Management

```
# Check GPU memory
helixcode-harmony device gpu-memory

# Clear GPU cache
helixcode-harmony device gpu-clear-cache
```

Model Optimization

Quantization

```yaml
harmony:
  ai_acceleration:
    enable_quantization: true
    quantization_mode: "dynamic"  # or "static"
    quantization_bits: 8
```

Model Caching

```yaml
harmony:
  ai_acceleration:
    model_cache:
      enabled: true
      directory: "/var/lib/helixcode/model_cache"
      max_size: "10GB"
      eviction_policy: "lru"
```

Multi-Screen Collaboration

Overview

Leverage Harmony OS multi-screen capabilities to work across multiple displays.

Enabling Multi-Screen

```yaml
harmony:
  enable_multi_screen: true
  multi_screen:
    auto_detect: true
    mirror_mode: false
    extend_mode: true
```

Screen Configuration

```
# List available screens
helixcode-harmony screens list

# Configure screen layout
helixcode-harmony screens layout \
  --primary 0 \
```

```
  --secondary 1 \
  --mode extend

# Launch on specific screen
helixcode-harmony --screen 1
```

Cross-Screen Features

Drag and Drop

Enabled by default. Drag tasks, files, or sessions between screens.

Screen Mirroring

```
# Mirror primary screen to secondary
helixcode-harmony screens mirror --source 0 --target 1
```

Independent Workspaces

```
harmony:
  multi_screen:
    independent_workspaces: true
    workspace_per_screen: true
```

Resource Management

Overview

The resource manager optimizes CPU, memory, GPU, and NPU usage automatically.

Configuration

```
harmony:
  enable_resource_optimization: true
  resource_manager:
    cpu_threshold: 80  # percent
    memory_threshold: 85  # percent
    gpu_threshold: 90  # percent

    # Optimization policies
    policy: "balanced"  # or "performance", "power_saving"
```

```
    # Scheduling
    task_priority_enabled: true
    preemption_enabled: true
```

Resource Policies

Balanced

Balances performance and power consumption:

```
harmony:
  resource_manager:
    policy: "balanced"
    cpu_governor: "ondemand"
    gpu_frequency: "auto"
```

Performance

Maximizes performance:

```
harmony:
  resource_manager:
    policy: "performance"
    cpu_governor: "performance"
    gpu_frequency: "max"
    npu_frequency: "max"
```

Power Saving

Prioritizes battery life:

```
harmony:
  resource_manager:
    policy: "power_saving"
    cpu_governor: "powersave"
    gpu_frequency: "min"
    background_tasks_limited: true
```

Monitoring Resources

```
# Real-time resource monitoring
helixcode-harmony resources monitor

# Resource usage history
helixcode-harmony resources history --duration 1h
```

```
# Resource alerts
helixcode-harmony resources alerts
```

Service Coordination

Overview

Service coordination manages distributed services with automatic discovery and failover.

Configuration

```yaml
harmony:
  service_coordination:
    service_discovery_enabled: true
    service_failover_enabled: true
    health_check_interval: 15

    # Service registry
    registry_type: "distributed"  # or "centralized"
    registry_port: 8083
```

Service Registration

```
# Register a service
helixcode-harmony service register \
  --name llm-service \
  --host localhost \
  --port 8000 \
  --health-endpoint /health

# List services
helixcode-harmony service list

# Check service health
helixcode-harmony service health llm-service
```

Failover Configuration

```yaml
harmony:
  service_coordination:
    failover:
```

```yaml
enabled: true
strategy: "active_passive"  # or "active_active"
heartbeat_interval: 5  # seconds
failover_timeout: 30  # seconds

# Health checks
health_check:
  interval: 10
  timeout: 5
  retries: 3
```

User Interface

Dashboard Tab

The dashboard provides an overview of:

- **System Status**: CPU, memory, GPU, NPU usage
- **Active Tasks**: Currently executing tasks with progress
- **Worker Pool**: Connected distributed workers
- **Recent Activity**: Last 10 operations
- **Alerts**: System warnings and errors

Tasks Tab

Task management interface with:

- **Task List**: All tasks with status, priority, and progress
- **Filter/Search**: Filter by status, type, priority
- **Task Details**: Detailed view of selected task
- **Actions**: Create, cancel, retry, delete tasks

Workers Tab

Worker pool management:

- **Worker List**: All connected workers with status
- **Worker Details**: Capabilities, resource usage
- **Connection Status**: Network health
- **Actions**: Add, remove, configure workers

Settings Tab

Configuration interface:

- **General**: Basic application settings
- **Distributed Computing**: Cluster configuration
- **AI Acceleration**: NPU/GPU settings
- **Synchronization**: Cross-device sync options
- **Theme**: Visual appearance customization
- **Advanced**: Expert settings

Theme Customization

Harmony Theme

The default Harmony theme uses warm colors optimized for Harmony OS:

- **Primary**: Warm Orange (#FF6B35)
- **Secondary**: Golden Orange (#F7931E)
- **Accent**: Light Amber (#FDB462)
- **Background**: Dark Warm Brown (#1A1512)

Switching Themes

```
# Via CLI
helixcode-harmony theme set harmony

# Available themes
helixcode-harmony theme list
# Output: Dark, Light, Helix, Harmony
```

Custom Theme

Create a custom theme:

```yaml
theme:
  name: "MyTheme"
  is_dark: true
  colors:
    primary: "#YOUR_COLOR"
    secondary: "#YOUR_COLOR"
    accent: "#YOUR_COLOR"
    text: "#FFFFFF"
```

```
  background: "#000000"
  border: "#333333"
```

Load custom theme:

```
helixcode-harmony theme load /path/to/theme.yaml
```

Troubleshooting

Common Issues

Issue: Application Won't Start

Symptoms: Service fails to start, immediate exit

Solutions:

1. Check configuration file syntax:

   ```
   helixcode-harmony config validate
   ```

2. Verify database connection:

   ```
   psql -h localhost -U helixcode -d helixcode
   ```

3. Check logs:

   ```
   sudo journalctl -u helixcode-harmony -n 50
   ```

4. Verify permissions:

   ```
   ls -la /opt/helixcode/harmony-os
   ls -la /var/log/helixcode
   ```

Issue: Distributed Computing Not Working

Symptoms: Workers not connecting, tasks not distributing

Solutions:

1. Check network connectivity:

   ```
   ping <worker-ip>
   telnet <worker-ip> 8081
   ```

2. Verify firewall rules:

```
sudo firewall-cmd --list-ports
# Ensure 8080-8083 are open
```

3. Check distributed engine status:

```
helixcode-harmony distributed status
```

4. Review worker logs:

```
helixcode-harmony distributed workers --verbose
```

Issue: NPU/GPU Not Detected

Symptoms: AI acceleration disabled, falling back to CPU

Solutions:

1. Check device availability:

```
helixcode-harmony device list
```

2. Verify drivers:

```
# For NPU
ls -la /dev/davinci*

# For GPU
nvidia-smi  # if applicable
```

3. Update configuration:

```
harmony:
  npu_enabled: true
  npu_config:
    device: "auto"  # Let system auto-detect
```

Issue: High Memory Usage

Symptoms: Application consuming excessive memory

Solutions:

1. Enable memory optimization:

```
harmony:
  resource_manager:
    memory_optimization: true
    max_memory: "4GB"
```

2. Clear caches:

   ```
   helixcode-harmony cache clear
   ```

3. Reduce concurrent tasks:

   ```yaml
   workers:
     max_concurrent_tasks: 10  # Reduce from 20
   ```

Diagnostic Commands

```
# System diagnostics
helixcode-harmony diagnostics run

# Health check
helixcode-harmony health

# Configuration test
helixcode-harmony config test

# Network test
helixcode-harmony network test

# Database connection test
helixcode-harmony db test
```

Debug Mode

Enable debug logging:

```yaml
logging:
  level: "debug"
  file: "/var/log/helixcode/harmony-debug.log"
```

Or via environment variable:

```
HELIX_LOG_LEVEL=debug helixcode-harmony
```

Best Practices

Performance Optimization

1. **Use Redis for Caching**

```yaml
redis:
  enabled: true
```

2. **Enable Resource Optimization**

```yaml
harmony:
  enable_resource_optimization: true
```

3. **Configure Task Priorities**

```yaml
tasks:
  priority_levels: ["low", "normal", "high", "critical"]
  default_priority: "normal"
```

4. **Optimize Checkpoint Intervals**

```yaml
tasks:
  checkpoint_interval: 300  # 5 minutes
  checkpoint_compression: true
```

Security Best Practices

1. **Use Strong Secrets**

```bash
# Generate secure secrets
openssl rand -hex 32
```

2. **Enable TLS**

```yaml
server:
  tls_enabled: true
  tls_cert: "/etc/helixcode/cert.pem"
  tls_key: "/etc/helixcode/key.pem"
```

3. **Restrict Network Access**

```yaml
server:
  allowed_ips:
    - "192.168.1.0/24"
```

4. **Regular Backups**

```bash
# Backup database
pg_dump helixcode > backup-$(date +%Y%m%d).sql

# Backup configuration
cp -r /etc/helixcode /backup/helixcode-config-$(date +%Y%m%d)
```

Distributed Computing Best Practices

1. **Balance Worker Load**
   - Use `load_balanced` distribution strategy
   - Monitor worker health regularly
   - Set appropriate task priorities

2. **Handle Failures Gracefully**

```yaml
tasks:
  max_retries: 3
  retry_delay: 60  # seconds
  enable_checkpointing: true
```

3. **Optimize Network Usage**

```yaml
harmony:
  distributed:
    compression_enabled: true
    batch_size: 10
```

Maintenance

1. **Regular Log Rotation**

```
# Configure logrotate
sudo nano /etc/logrotate.d/helixcode

/var/log/helixcode/*.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 0644 helixcode helixcode
}
```

2. **Monitor Disk Usage**

```
du -sh /var/lib/helixcode/*
```

3. **Database Maintenance**

```
-- Vacuum database
VACUUM ANALYZE;
```

```
-- Reindex
REINDEX DATABASE helixcode;
```

FAQ

General Questions

Q: What is the difference between Harmony OS client and standard desktop client?

A: The Harmony OS client includes specialized features: - Distributed computing engine for multi-device coordination - NPU/GPU acceleration support - Cross-device synchronization via Super Device - Multi-screen collaboration - Harmony OS-specific optimizations

Q: Can I run the Harmony OS client on non-Harmony OS systems?

A: Yes, but some features will be unavailable: - NPU acceleration requires Kirin NPU hardware - Super Device features require Harmony OS ecosystem - Multi-screen may have limited functionality

Q: How much resources does the client consume?

A: Typical resource usage: - RAM: 45-65 MB idle, up to 200 MB under load - CPU: 5-15% average, 30-50% during AI tasks - Storage: ~56 MB binary + caches and logs

Distributed Computing

Q: How many workers can I connect?

A: The system supports up to 100 workers by default. This can be increased in configuration.

Q: Do workers need to be on the same network?

A: No, workers can be on different networks. Ensure proper firewall rules and port forwarding.

Q: What happens if a worker goes offline during a task?

A: The task is automatically reassigned to another worker. Progress is preserved via checkpoints.

AI Acceleration

Q: Which NPU models are supported?

A: Kirin 990, 9000 series and later NPUs are supported.

Q: Can I use both NPU and GPU simultaneously?

A: Yes, you can enable both. The system will intelligently route tasks to the most appropriate accelerator.

Q: How much faster is NPU vs CPU inference?

A: Typically 5-10x faster for supported model architectures, with lower power consumption.

Synchronization

Q: What data is synchronized across devices?

A: By default: tasks, sessions, and configurations. Logs and caches are optional.

Q: How is data encrypted during sync?

A: All data is encrypted in transit using TLS 1.3 and at rest using AES-256.

Q: Can I selectively sync certain data?

A: Yes, configure `sync_items` in the configuration file.

Troubleshooting

Q: Why is my NPU not being detected?

A: Common causes: - Outdated Harmony OS version (requires 3.0+) - Missing NPU drivers - Incorrect configuration - NPU already in use by another application

Q: Tasks are stuck in “pending” status

A: Check: - Worker availability: `helixcode-harmony workers list` - Task queue: `helixcode-harmony task queue` - Database connectivity - System resources

Q: How do I reset to factory defaults?

A:

```
sudo systemctl stop helixcode-harmony
sudo rm -rf /var/lib/helixcode/*
```

```
sudo cp /etc/helixcode/harmony-config.yaml.default /etc/
        helixcode/harmony-config.yaml
sudo systemctl start helixcode-harmony
```

Additional Resources

Documentation

- API Reference
- Architecture Guide
- Configuration Reference
- Development Guide

Support

- **GitHub Issues**: https://github.com/helixcode/helixcode/issues
- **Community Forum**: https://forum.helixcode.dev
- **Documentation**: https://docs.helixcode.dev

Related Guides

- Aurora OS Guide - For Aurora OS platform
- Quick Start Guide - Getting started quickly
- Deployment Guide - Production deployment

Document Version: 1.0.0 **Last Updated**: 2025-11-07 **HelixCode Version**: 1.0.0+